Como adicionar login com SUAP a uma aplicação Flask

OAuth 2.0 · Authorization Code · Flask-Login



Guia técnico de configuração e implementação

1. Objetivo e escopo da integração

Este tutorial apresenta a incorporação do mecanismo de autenticação do SUAP a uma aplicação Flask. O processo disponibiliza o recurso Entrar com SUAP, redireciona o usuário para o provedor de identidade, recebe um código temporário de autorização, realiza a troca desse código por um token e consulta os dados do perfil para estabelecer o vínculo com o usuário local.

> Princípio de segurança: a senha é informada exclusivamente ao SUAP. A aplicação Flask não recebe, não processa e não armazena essa credencial.

Resultados esperados

Resultados esperados

• O login local continua funcionando.

• A tela de login ganha um botão para entrar pelo SUAP.

• O perfil retornado pelo SUAP é associado ao modelo local Usuario.

• O Flask-Login inicia a sessão.

• Uma página exibe os dados do perfil.

Arquivos envolvidos

Arquivos envolvidos

Arquivo	Responsabilidade
config.py	Lê URL, client ID, segredo e chave de sessão.
app/models/usuario.py	Guarda o identificador que vincula usuário local e SUAP.
app/services/SuapAuthenticationService.py	Monta autorização, troca código por token e consulta API.
app/services/AuthenticationService.py	Localiza/cria a conta e chama login_user.
app/routes.py	Inicia o fluxo e recebe o callback.
app/templates/login.html	Apresenta o botão.
app/templates/suap_eu.html	Mostra o perfil.

2. Fluxo de autenticação OAuth 2.0



Figura 1 — Fluxo Authorization Code usado na integração.

• 1. A aplicação gera um state aleatório e o guarda na sessão.

• 2. O navegador é redirecionado para /o/authorize/ no SUAP.

• 3. A pessoa faz login e autoriza o acesso.

• 4. O SUAP retorna à aplicação com code e state.

• 5. O servidor confere o state e envia o code para /o/token/.

• 6. O SUAP devolve um access_token.

• 7. O Flask consulta /api/rh/meus-dados/ usando Bearer token.

• 8. Os dados identificam ou criam o usuário local.

> Finalidade do parâmetro state: estabelecer a correspondência entre a solicitação de autenticação e o retorno à sessão que a originou. Caso os valores sejam distintos, o callback deve ser rejeitado.

O client_secret aparece somente entre o servidor Flask e o SUAP. Ele nunca deve ser colocado em HTML, JavaScript do navegador, repositório ou logs.

3. Preparação do ambiente de execução

3.1 Criar o ambiente Python

3.1 Criar o ambiente Python

```
python -m venv .venv
source .venv/bin/activate          # Linux/macOS
# .venv\Scripts\activate            # Windows
python -m pip install --upgrade pip
pip install -r requirements.txt
```

O projeto usa Flask, Flask-Login, Flask-SQLAlchemy, Flask-Migrate, python-dotenv e o driver do MySQL.

3.2 Criar o banco

3.2 Criar o banco

```
mysql -u root -p
CREATE DATABASE suap_flask_app CHARACTER SET utf8mb4;
EXIT;
```

Devem ser configurados, em config.py, o nome do banco de dados, o usuário, a senha, o endereço do servidor e a porta de comunicação. A implementação utiliza o driver mysql+mysqlconnector.

3.3 Configurar variáveis de ambiente

3.3 Configurar variáveis de ambiente

Deve ser criado um arquivo .env no diretório raiz do projeto:

```
SECRET_KEY=uma-chave-aleatoria-e-secreta
DB_USERNAME=root
DB_PASSWORD=sua_senha_do_mysql
SUAP_URL=https://suap.ifrn.edu.br
SUAP_CLIENT_ID=seu_client_id
SUAP_CLIENT_SECRET=seu_client_secret
```

Recomenda-se gerar uma chave criptograficamente forte por meio do comando:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

> As credenciais não devem ser versionadas. Caso um segredo seja exposto, ele deve ser revogado no painel do SUAP e substituído imediatamente.

4. Registro da aplicação no SUAP

O registro da aplicação fornece o Client ID e o Client Secret. Embora a nomenclatura da interface possa variar, os parâmetros devem obedecer às seguintes definições:

A documentação oficial da API do SUAP deve ser consultada para acessar a área de criação e configuração da aplicação OAuth2:

```
https://suap.ifrn.edu.br/api/docs/
```

Nessa documentação, localize a seção relacionada à autenticação OAuth2 e utilize o painel correspondente para registrar o cliente, definir o tipo Confidential, selecionar o fluxo Authorization Code e informar a URI de redirecionamento.

• Acesse a área de APIs ou aplicações OAuth e autentique-se.

• Registre uma nova aplicação.

• Selecione o tipo de cliente Confidential, pois o segredo será mantido no servidor.

• Selecione o grant Authorization Code.

• Informe a URI local:

```
http://127.0.0.1:5000/suap/callback
```

• Salve o registro e transfira as credenciais para o arquivo .env.

> A URI deve ser idêntica. Protocolo, host, porta, caminho e barra final fazem parte da validação. localhost e 127.0.0.1 não devem ser tratados como equivalentes.

Configuração para produção

Configuração para produção

Para o ambiente de produção, deve ser registrada uma URI HTTPS específica, por exemplo:

```
https://app.exemplo.org/suap/callback
```

Devem ser utilizadas credenciais distintas em cada ambiente; o segredo empregado no desenvolvimento não deve ser reutilizado em produção.

5. Atualização do modelo e do banco de dados

5.1 Adicionar o identificador SUAP

5.1 Adicionar o identificador SUAP

Na classe Usuario, deve ser acrescentada uma coluna opcional e de valor único:

```
class Usuario(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), index=True, unique=True, nullable=False)
    email = db.Column(db.String(64), index=True, unique=True, nullable=False)
    password_hash = db.Column(db.String(256))
    suap_id = db.Column(db.String(64), unique=True, nullable=True)
    posts = db.relationship('Post', back_populates='author')
```

nullable=True preserva as contas antigas, que ainda não possuem vínculo SUAP.

5.2 Gerar e aplicar a migration

5.2 Gerar e aplicar a migration

```
flask db migrate -m "adiciona identificador SUAP ao usuario"
flask db upgrade
flask db current
```

Deve-se verificar o arquivo de migration em migrations/versions/, confirmando a criação de suap_id com restrição de unicidade. Caso a migration já esteja presente no projeto, deve ser executado somente flask db upgrade.

> Não execute flask db init em um projeto que já possui a pasta migrations.

5.3 Revisão ausente

5.3 Revisão ausente

Se for apresentada a mensagem “Can't locate revision”, o banco de dados estará referenciando uma revisão ausente nos arquivos locais. A migration correspondente deverá ser recuperada antes da atualização. A tabela alembic_version não deve ser alterada manualmente sem a confirmação da compatibilidade do schema.

6. Implementação do serviço de comunicação

O arquivo app/services/SuapAuthenticationService.py concentra as requisições HTTP destinadas ao SUAP.

6.1 Construção da URL de autorização

6.1 Construção da URL de autorização

```
@staticmethod
def authorization_url(state, redirect_uri):
    params = urlencode({
        'response_type': 'code',
        'client_id': app.config['SUAP_CLIENT_ID'],
        'redirect_uri': redirect_uri,
        'state': state,
    })
    return f"{app.config['SUAP_URL']}/o/authorize/?{params}"
```

A URL é encaminhada ao navegador. A aplicação informa sua identidade e o endereço de retorno, sem transmitir a senha do usuário.

6.2 Troca do código por token

6.2 Troca do código por token

```
data = urlencode({
    'grant_type': 'authorization_code',
    'code': code,
    'redirect_uri': redirect_uri,
    'client_id': app.config['SUAP_CLIENT_ID'],
    'client_secret': app.config['SUAP_CLIENT_SECRET'],
}).encode()

with urlopen(Request(
    f"{app.config['SUAP_URL']}/o/token/",
    data=data,
    headers={'Content-Type': 'application/x-www-form-urlencoded'},
)) as response:
    return json.load(response)
```

O código possui validade temporária e utilização única. Em caso de falha, o serviço retorna None.

6.3 Consulta ao perfil

6.3 Consulta ao perfil

```
request = Request(
    f"{app.config['SUAP_URL']}/api/rh/meus-dados/",
    headers={
        'Authorization': f'Bearer {access_token}',
        'Accept': 'application/json',
    },
)
```

O token é enviado no cabeçalho HTTP Authorization: Bearer ....

7. Implementação das rotas Flask

7.1 Rota de início da autenticação

7.1 Rota de início da autenticação

```
@app.route('/login-suap')
def login_suap():
    state = token_urlsafe(32)
    redirect_uri = request.host_url.rstrip('/') + url_for('suap_callback')
    session['suap_oauth_state'] = state
    return redirect(SuapAuthenticationService.authorization_url(state, redirect_uri))
```

A URI de callback é construída a partir do host da requisição. Consequentemente, o host e a porta registrados no SUAP devem coincidir com o endereço utilizado pelo navegador.

7.2 Rota de retorno (callback)

7.2 Rota de retorno (callback)

```
@app.route('/suap/callback')
def suap_callback():
    state = session.pop('suap_oauth_state', None)
    if request.args.get('error') or not state \
       or state != request.args.get('state', ''):
        flash('Falha na autenticação do SUAP.', 'error')
        return redirect(url_for('login'))

    code = request.args.get('code')
    if not code:
        flash('Código de autorização não recebido.', 'error')
        return redirect(url_for('login'))

    redirect_uri = request.host_url.rstrip('/') + url_for('suap_callback')
    tokens = SuapAuthenticationService.exchange_code(code, redirect_uri)
    dados = SuapAuthenticationService.meus_dados(tokens['access_token'])
    usuario = AuthenticationService.login_suap(dados)
```

Sequência de processamento: validar state -> validar code -> trocar o código -> consultar o perfil -> vincular a conta -> iniciar a sessão.

8. Vinculação da identidade SUAP ao usuário local

Em AuthenticationService.login_suap, a aplicação determina o identificador do usuário segundo a seguinte precedência: matrícula, username ou endereço de e-mail.

```
suap_id = str(
    dados.get('matricula') or
    dados.get('username') or
    dados.get('email') or ''
).strip()

if not suap_id:
    return None

email = dados.get('email') or f'{suap_id}@suap.local'
user = Usuario.query.filter_by(suap_id=suap_id).first()

if not user:
    user = Usuario.query.filter(
        (Usuario.username == suap_id) | (Usuario.email == email)
    ).first()
```

Critérios de vinculação

Critérios de vinculação

• Pesquisa-se inicialmente um usuário que já possua o suap_id.

• Na ausência desse registro, pesquisa-se uma conta com username ou e-mail compatível.

• Não havendo correspondência, cria-se um novo objeto Usuario.

• A associação é persistida e login_user(user) inicia a sessão.

```
if not user:
    user = Usuario(username=suap_id, email=email, suap_id=suap_id)
    db.session.add(user)
else:
    user.suap_id = suap_id

db.session.commit()
login_user(user)
return user
```

> Não copie a senha do SUAP. A confiança vem do token e do perfil retornado pelo SUAP.

9. Integração da interface de usuário

9.1 Botão na tela de login

9.1 Botão na tela de login

Em app/templates/login.html, deve ser incluído um link para a rota que inicia o fluxo OAuth:

```
<a href="{{ url_for('login_suap') }}"
   class="btn btn-secondary btn-user btn-block">
    Entrar com SUAP
</a>
```

O botão não realiza a troca do token no navegador; sua única finalidade é iniciar a rota Flask.

9.2 Página de perfil

9.2 Página de perfil

A rota encaminha o dicionário dados ao template suap_eu.html. O template apresenta nome, tipo de vínculo, matrícula, e-mail, campus, curso ou cargo, situação e fotografia, quando esses atributos estiverem disponíveis.

```
{% set vinculo = dados.get('vinculo') %}
<h2>{{ dados.get('nome_usual') or dados.get('nome') or 'Usuário SUAP' }}</h2>
<p>{{ dados.get('email') or 'Não informado' }}</p>
```

Recomenda-se utilizar get() e valores alternativos, uma vez que os atributos retornados pela API podem variar conforme o tipo de vínculo.

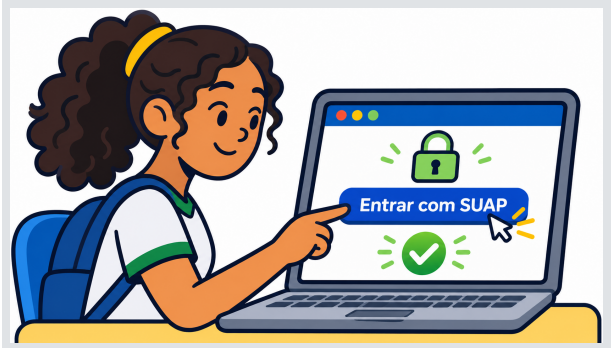


Figura 2 — A autenticação acontece na tela do SUAP.

10. Execução e validação do fluxo

10.1 Inicialização da aplicação

10.1 Inicialização da aplicação

```
flask db upgrade
flask run --debug
```

Acesse http://127.0.0.1:5000/login e execute o seguinte procedimento:

• Verifique a presença do botão.

• Selecione Entrar com SUAP.

• Autentique-se no SUAP.

• Autorize a aplicação.

• Observe o retorno para /suap/callback.

• Confirme a apresentação dos dados do perfil.

• Verifique no banco de dados o preenchimento de suap_id.

10.2 Comportamento esperado

10.2 Comportamento esperado

Etapa	Resultado
Login	Redirecionamento para o domínio do SUAP.
Autorização	A senha aparece somente no SUAP.
Callback	A URL recebe code e state temporários.
Perfil	A aplicação exibe os dados da API.
Segundo login	O mesmo usuário é encontrado pelo suap_id.

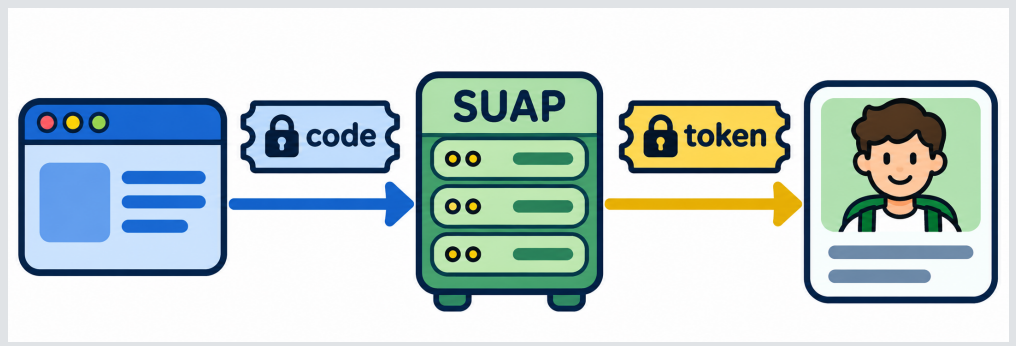


Figura 3 — O código é trocado por token no servidor.

11. Diagnóstico e resolução de problemas

“redirect_uri mismatch”

“redirect_uri mismatch”

Compare o registro no SUAP com request.host_url.rstrip('/') + url_for('suap_callback'). Devem coincidir o protocolo, o host, a porta e o caminho.

“state” inválido

“state” inválido

A sessão pode não estar sendo preservada ou o callback pode ter sido acessado diretamente. O fluxo deve ser iniciado pelo botão e a SECRET_KEY deve permanecer estável.

Token não obtido

Token não obtido

Verifique o Client ID, o Client Secret, o grant, a URI de callback e a eventual reutilização do código. O código possui validade temporária e uso único.

Perfil não retornado

Perfil não retornado

Verifique o cabeçalho Bearer e a URL da API. Tokens não devem ser registrados em logs.

A autenticação ocorre, mas a sessão não permanece

A autenticação ocorre, mas a sessão não permanece

Verifique login_user(user), o user_loader em app/__init__.py, os cookies e a SECRET_KEY. As rotas protegidas devem utilizar @login_required.

Erro de migration

Erro de migration

Execute flask db history e flask db heads. Se o banco referenciar uma revisão ausente localmente, recupere o arquivo correspondente antes de alterar versões.

> Checklist de conformidade: credenciais fora do Git; URI de callback exata; state validado; segredo restrito ao servidor; token não persistido sem necessidade; suap_id único; tratamento explícito de erros.

12. Funções completas usadas no login SUAP

Esta seção reúne as funções de autenticação implementadas no projeto. Os exemplos mantêm a lógica e a ordem de operações da implementação analisada.

12.1 Serviço de autenticação SUAP

12.1 Serviço de autenticação SUAP

```
class SuapAuthenticationService:
    @staticmethod
    def authorization_url(state, redirect_uri):
        params = urlencode({
            'response_type': 'code',
            'client_id': app.config['SUAP_CLIENT_ID'],
            'redirect_uri': redirect_uri,
            'state': state,
        })
        return f"{app.config['SUAP_URL']}/o/authorize/?{params}"

    @staticmethod
    def exchange_code(code, redirect_uri):
        data = urlencode({
            'grant_type': 'authorization_code',
            'code': code,
            'redirect_uri': redirect_uri,
            'client_id': app.config['SUAP_CLIENT_ID'],
            'client_secret': app.config['SUAP_CLIENT_SECRET'],
        }).encode()
        try:
            with urlopen(Request(
                f"{app.config['SUAP_URL']}/o/token/",
                data=data,
                headers={'Content-Type': 'application/x-www-form-urlencoded'},
            )) as response:
                return json.load(response)
        except URLError:
            return None

    @staticmethod
    def meus_dados(access_token):
        request = Request(
            f"{app.config['SUAP_URL']}/api/rh/meus-dados/",
            headers={
                'Authorization': f'Bearer {access_token}',
                'Accept': 'application/json',
            },
        )
        try:
            with urlopen(request) as response:
                return json.load(response)
        except URLError:
            return None
```

12.2 Rotas que iniciam e concluem o OAuth

12.2 Rotas que iniciam e concluem o OAuth

12. Funções completas usadas no login SUAP (continuação

```
@app.route('/login-suap')
def login_suap():
    state = token_urlsafe(32)
    redirect_uri = request.host_url.rstrip('/') + url_for('suap_callback')
    session['suap_oauth_state'] = state
    return redirect(SuapAuthenticationService.authorization_url(state, redirect_uri))

@app.route('/suap/callback')
def suap_callback():
    state = session.pop('suap_oauth_state', None)
    if request.args.get('error') or not state or state != request.args.get('state', ''):
        flash('Falha na autenticação do SUAP.', 'error')
        return redirect(url_for('login'))

    code = request.args.get('code')
    if not code:
        flash('Código de autorização do SUAP não recebido.', 'error')
        return redirect(url_for('login'))

    redirect_uri = request.host_url.rstrip('/') + url_for('suap_callback')
    tokens = SuapAuthenticationService.exchange_code(code, redirect_uri)
    if not tokens or not tokens.get('access_token'):
        flash('Não foi possível obter o token do SUAP.', 'error')
        return redirect(url_for('login'))

    dados = SuapAuthenticationService.meus_dados(tokens['access_token'])
    if dados is None:
        flash('Não foi possível consultar os dados do SUAP.', 'error')
        return redirect(url_for('login'))

    usuario = AuthenticationService.login_suap(dados)
    if usuario is None:
        flash('Não foi possível vincular esta conta SUAP a um usuário local.', 'error')
        return redirect(url_for('login'))

    foto = dados.get('url_foto_75x100') or dados.get('url_foto') or dados.get('foto')
    foto_url = urljoin(app.config['SUAP_URL'] + '/', foto) if foto else None
    return render_template('suap_eu.html', title='Perfil SUAP', dados=dados, foto_url=fo
to_url)
```

12.3 Vínculo com o usuário local

12.3 Vínculo com o usuário local

12. Funções completas usadas no login SUAP (continuação

```
def login_suap(dados):
    suap_id = str(dados.get('matricula') or dados.get('username') or dados.get('email')
or '').strip()
    if not suap_id:
        return None

    username = suap_id
    email = dados.get('email') or f'{username}@suap.local'
    user = Usuario.query.filter_by(suap_id=suap_id).first()

    if not user:
        user = Usuario.query.filter((Usuario.username == username) | (Usuario.email == e
mail)).first()

    if user and user.suap_id and user.suap_id != suap_id:
        return None

    if not user:
        user = Usuario(username=username, email=email, suap_id=suap_id)
        db.session.add(user)
    else:
        user.suap_id = suap_id

    try:
        db.session.commit()
    except Exception:
        db.session.rollback()
        return None

    login_user(user)
    return user
```

13. Síntese da implementação

A integração do login com o SUAP articula cinco componentes principais:

• Configuração: URL, Client ID, Client Secret e SECRET_KEY.

• Banco: coluna opcional e única suap_id.

• Serviço: autorização, troca do código e consulta da API.

• Rotas: início, callback, validações e erros.

• Sessão: localizar/criar usuário e chamar Flask-Login.

> Síntese do fluxo: botão Flask -> autorização no SUAP -> callback com code/state -> obtenção do token no servidor -> consulta ao perfil -> vínculo local -> sessão Flask-Login.

Recomendações de segurança e manutenção

Recomendações de segurança e manutenção

• Utilizar HTTPS em qualquer ambiente acessível pela rede pública.

• Armazenar segredos em variáveis de ambiente ou em um gerenciador de segredos.

• Evitar o registro de senhas, client secrets, access tokens e dados pessoais desnecessários nos logs.

• Implementar testes para state inválido, callback sem code, falha na obtenção do token e usuário previamente vinculado.

• Priorizar um identificador permanente do SUAP, caso a API o disponibilize.

Tutorial genérico baseado na estrutura atual de uma aplicação Flask com Flask-Login, SQLAlchemy, Flask-Migrate e integração OAuth 2.0 com o SUAP. Adapte endpoints, campos e URLs conforme a documentação e o ambiente do seu SUAP.